Slides for P3776R1 — More trailing commas

Document number:
D3897R0

Date:
2025-10-31

Audience:
EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/more-trailing-commas-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

More trailing commas

P3776R1

Introduction

```
void f(
  int x,
  int y, // ← trailing comma should be allowed here
);

int x[] { 1, 2, }; // already supported in braced lists
```

- trailing commas would be useful
- [P0562R2] Trailing Commas in Base-clauses and Ctor-initializers

P0562R2

> Poll: D0562R1 — forward [...] to CWG for inclusion in C++26.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 12 | 11 | 4 | 3 | 0 |

Forwarded to CWG, but currently on ice because of parsing ambiguity:

```
struct X {};
struct Y : X,      // ← OK, new trailing comma
{ Y() : A<b<c>(), // ← parsing ambiguity due to trailing comma
  { /* ... */ }
  /* A, b, and c declared down here somewhere */ };
```

Trailing commas in other languages

Many languages now support trailing commas in parenthesized lists:

Swift
2025

Kotlin
2020

JavaScript
2017

TypeScript
2017

Rust
always

Go
always

Python
always

Julia
always

Many more (including C++) support trailing commas in braced lists.

Improved text editing

Without trailing commas, we cannot always swap or cut/paste lines:

No trailing comma	With trailing comma

`// before`
`void f(`
`  int x,`
`  int y`
`);`
`// after`
`void f(`
`  int y  // syntax error`
`  int x,`
`);`
	`// before`
	`void f(`
	`  int x,`
	`  int y,`
	`);`
	`// after`
	`void f(`
	`  int y, // OK`
	`  int x,`
	`);`

Improved version control

No trailing comma	With trailing comma

`void f(`
`    int x,`
`-    int y`
`+    int y,`
`+    int z`
`);`
	`void f(`
	`    int x,`
	`    int y,`
	`+    int z,`
	`);`

- trailing comma is much better for version control:
  - smaller changes when viewing line differences
  - no pollution of per-line history (`git blame`)

Code generation convenience for [P3294R2]

No trailing comma	With trailing comma

`consteval meta::info gen_params(`
`  span<const Parameter> ps`
`) {`
`  meta::info result = ^^{};`
`  bool first = true;`
`  for (const Parameter& p : ps) {`
`  result = ^^{`
`  \tokens(result)`
`  \tokens(first ? ^^{} : ^^{,})`
`  \tokens(gen_param(p))`
`  };`
`  first = false;`
`  }`
`  return result;`
`}`
	`consteval meta::info gen_params(`
	`  span<const Parameter> ps,`
	`) {`
	`  meta::info result = ^^{};`
	`  for (const Parameter& p : ps) {`
	`  result = ^^{`
	`  \tokens(result)`
	`  \tokens(gen_param(p)) ,`
	`  };`
	`  }`
	`  return result;`
	`}`

Improved auto-formatter control

- ClangFormat disables bin-packing when trailing comma is present
- this allows fine-tuned stylistic control via comma

```
vector<int> numbers { LIST_ITEM_A, LIST_ITEM_B,  //
                      LIST_ITEM_C };             // ← column limit
                                                 //
vector<int> numbers {
    LIST_ITEM_A,
    LIST_ITEM_B,
    LIST_ITEM_C, // ← trailing comma here
};
```

Eliminating some uses of __VA_OPT__

No trailing comma	With trailing comma

`#define F(...)  \`
`  f(0 __VA_OPT__(,)  \`
`  __VA_ARGS__)`
 	
`#define L(...)  \`
`  [& __VA_OPT__(,)  \`
`  __VA_ARGS__] {}`
	`#define F(...)  \`
	`  f(0 , __VA_ARGS__)`
	
	`#define L(...)  \`
	`  [& , __VA_ARGS__] {}`

- `__VA_OPT__` is necessary to avoid error on trailing comma
- no longer necessary if trailing comma is OK

Preventing string joining bugs

No trailing comma	With trailing comma

`emplace_strings(`
`  "fee",`
`  "fie",`
`  "foe"`
`+  "fum"`
`);`
	`emplace_strings(`
	`  "fee",`
	`  "fie",`
	`  "foe",`
	`+  "fum"`
	`);`

- no trailing comma: `"foe"` and `"fum"` are concatenated
- missing commas happen (forgetfulness, key slip, etc.)

Where to add trailing commas

- in summary: after all lists, except:
  - [P0562R2]'s *mem-initializer-list* and *base-specifier-list*
  - semicolon-terminated lists like `using a, b,;`
  - non-lists like `static_assert(true,"",)`, `delete("",)`
  - after ellipsis parameter like `void f(int, ...,)`
  - in macros like `#define M(x,y,)`
- see proposal for examples and more details

TODO: Discuss with clang-format maintainers.

Addressing criticisms (1/2)

- aesthetic objections
  - valid, although trailing commas are 100% optional
- asymmetry with C
  - just don't use the trailing comma if you need C compatibility
  - WG14 proposal will be published to synchronize languages
- making `f(0,)` valid may invite "forgotten argument" bugs
  - experience from other langs, `{0,}` in C++ tells us it's okay
  - type system, compiler warnings, auto-formatting catches this

Addressing criticisms (2/2)

- inconsistency with macros: `M(1,2,)` may be ill-formed
  - does not apply to variadic, i.e. `__VA_ARGS__` macros
  - macros are already inconsistent, e.g. `M({1,2})`
- multiple ways to do the same thing
  - that's okay, same for indentation and other editorial choices
  - similar to `//...` vs. `/*...*/`
- syntax space is claimed
  - no plausible alternative meaning for `f(1,)` exists anyway

Last but not least,

- **Implementation experience**: Clang fork,
  - approximately 40 LOC,
  - naive implementation that ignores trailing comma,
  - proper implementation may need changes to AST,
- **Impact on existing code**: None,
- **Wording**: Exists, briefly seen by CWG chair,

,

Questions

- Agreement with overall design?
- SIMD overloads in another proposal?
- Forward to LEWG?

References

[P3776R1] *Jan Schultke.* More trailing commas (2025-09-09)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3776r1.html

[P0562R2] *Alan Talbot.* Trailing Commas in Base-clauses and Ctor-initializers (2024-04-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p0562r2.pdf

[P3294R2] *Andrei Alexandrescu, Barry Revzin, Daveed Vandevoorde.* Code Injection with Token Sequences (2024-10-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3294r2.html